COMPLETE JAVASCRIPT PROGRAMMING SERIES

MODULE 9

OBJECT-ORIENTED JAVASCRIPT

Authors: Mithila Rani Saha & Dithsa Dutta

IT STUDY HUB

www.itstudyhub.dpdns.org

About This Module

Welcome to Volume 9 of the IT Study Hub Complete JavaScript Programming Series. You possess all the logical, DOM, and asynchronous skills needed to build applications. However, as your applications grow from 100 lines of code to 100,000 lines of code, standard variables and functions will collapse into a chaotic, unmaintainable mess.

To survive at scale, you must adopt the architectural blueprint of enterprise software: **Object-Oriented Programming (OOP)**. In this module, you will abandon procedural code. You will learn to model the real world by fusing data and behavior into modular `classes`. You will dissect JavaScript's hidden **Prototype Chain**, inherit traits like genetic trees, and lock down your application's security using true **Encapsulation**.

LEARNING OUTCOMES

- Transition from Procedural thought patterns to Object-Oriented paradigms.
- Master both legacy Constructor Functions and the modern ES6 `class` syntax.
- Understand Memory mapping via Prototypes and the Prototype Chain.
- Construct architectural blueprints utilizing Constructors, Instance Methods, and Static Methods.
- Implement deep Inheritance using `extends` and route data securely with `super`.
- Protect application integrity using Getters, Setters, and ES2022 Private Fields (#).
- Evaluate Object-Oriented Polymorphism against functional Composition structures.

MODULE COVERAGE

Topic	Description
1. OOP Core Mechanics	OOP concepts, Objects, Classes, Constructors
2. Prototype Architecture	Constructor functions, Prototypes, Prototype chain

IT STUDY HUB

3. Modern ES6 Classes	Instance methods, Static methods, Getters/setters
4. Hierarchical Design	Inheritance, extends, super, Polymorphism
5. Advanced State	Encapsulation, Private fields, Composition vs inheritance

IT STUDY HUB

OOP concepts

1. CONCEPT

> **Object-Oriented Programming (OOP) is a programming paradigm based on the concept of 'objects', which can contain data (attributes) and code (methods).**

2. SIMPLE ANALOGY

Think of OOP like building a real-world city. Instead of a long, confusing list of instructions on how to run the city (Procedural Programming), you create specific entities: Cars, Buildings, and People (Objects). Each entity knows its own details and how it should behave.

3. SYNTAX

```
// Conceptual shift from 'functions' to 'objects'.
```

4. EASY CODE

```
// Procedural (Old Way)
let carMake = "Ford";
let carSpeed = 0;
function driveCar() { carSpeed = 50; }

// OOP (Modern Way)
const car = {
    make: "Ford",
    speed: 0,
    drive() { this.speed = 50; }
};
```

5. OUTPUT

```
Output:
(Conceptual grouping of data and behavior)
```

6. EXPLANATION

In the procedural way, variables and functions float around loosely. In the OOP way, the data (`make`, `speed`) and the behavior (`drive()`) are grouped together into one logical, self-contained unit: the `car` object.

7. REAL-WORLD USE

Designing an entire video game. You create a 'Player' object, 'Enemy' objects, and 'Weapon' objects, each managing their own health and damage.

8. COMMON MISTAKES

Caution: Forgetting that OOP is not a specific feature of JavaScript, but a universal mental model used in Python, Java, C++, and almost every modern language.

9. PRACTICE

Task: Write an object `book` that has a `title` and a `read()` method.

Objects and classes

1. CONCEPT

> **A Class is a blueprint or template. An Object is the actual, physical instance built from that blueprint.**

2. SIMPLE ANALOGY

A Class is the architectural blueprint for a house. You cannot live inside a blueprint. An Object is the actual wooden house built using that blueprint. You can use the same blueprint to build 100 identical houses (100 objects).

3. SYNTAX

```
class Blueprint { }
const actualInstance = new Blueprint();
```

4. EASY CODE

```
class Student {
    // Blueprint logic goes here
}

// Creating two distinct objects from the same blueprint
const student1 = new Student();
const student2 = new Student();

console.log(student1);
console.log(typeof Student);
```

5. OUTPUT

```
Output:
Student {}
function
```

6. EXPLANATION

We created an empty blueprint called `Student`. We then used the `new` keyword to stamp out two distinct physical objects. Even though classes are a modern feature, JavaScript still secretly considers them 'functions' behind the scenes.

7. REAL-WORLD USE

A social media platform having one single `User` class blueprint, and stamping out 2 billion different User objects to represent every person on earth.

8. COMMON MISTAKES

Caution: Trying to add data directly to the Class blueprint itself. Data must be added to the instantiated Objects, not the blueprint.

9. PRACTICE

Task: Define an empty class called `Employee` and create one instance called `employee1`.

Constructor functions

1. CONCEPT

> **Before the 'class' keyword was added to JavaScript, developers used standard functions as blueprints. These are called Constructor Functions.**

2. SIMPLE ANALOGY

Constructor functions are the 'old-school' factory machines. You pass raw materials (arguments) into the machine, it uses the `this` keyword to assemble the parts, and spits out a finished product.

3. SYNTAX

```
function Car(brand) { this.brand = brand; }
```

4. EASY CODE

```
// Constructor function (Capitalized by convention)
function Employee(name, position) {
    this.name = name;
    this.position = position;
    this.work = function() {
        console.log(`${this.name} is working.`);
    };
}

// The 'new' keyword creates a blank object, and sets 'this' to point to it
const emp1 = new Employee("Mithila", "Developer");
const emp2 = new Employee("Dithsa", "Designer");

emp1.work();
```

5. OUTPUT

```
Output:
Mithila is working.
```

6. EXPLANATION

The `new` keyword does three invisible things: 1. Creates an empty object `{}`. 2. Points the `this` keyword to that object. 3. Returns the object automatically. Our constructor then attaches the `name` and `position`.

7. REAL-WORLD USE

You will see this constantly when maintaining older legacy JavaScript codebases written before ES6 (2015).

8. COMMON MISTAKES

Caution: Forgetting the `new` keyword when calling a constructor. If you write `const emp = Employee('John');`, it will fail to create an object and accidentally pollute the global window with variables.

9. PRACTICE

Task: Write a constructor function for `Book(title)` that attaches the title using `this`.

Prototypes

1. CONCEPT

> **A Prototype is an invisible parent object attached to every single JavaScript object, serving as a fallback location for shared properties and methods.**

2. SIMPLE ANALOGY

If you ask an employee for the company manual, and they don't have it on their desk (the object), they will naturally walk to the manager's office (the Prototype) to find it. Prototypes allow objects to share resources.

3. SYNTAX

```
Constructor.prototype.methodName = function() { };
```

4. EASY CODE

```
function User(name) {
    this.name = name;
}

// Attaching a method to the PROTOTYPE, not the object directly
User.prototype.login = function() {
    console.log(`${this.name} logged in securely.`);
};

const user1 = new User("Rahul");
const user2 = new User("Sara");

// Both objects share the exact same login function memory space!
user1.login();
```

5. OUTPUT

```
Output:
Rahul logged in securely.
```

6. EXPLANATION

If we put `login()` inside the constructor, copying 10,000 users means creating 10,000 copies of the exact same function. By putting it on the Prototype, we create ONE function that all 10,000 users share. This saves massive amounts of RAM.

7. REAL-WORLD USE

Every array you create automatically inherits `.push()` and `.map()` from the global `Array.prototype`.

8. COMMON MISTAKES

Caution: Adding arrays or objects directly to a prototype. If you put `User.prototype.friends = []`, ALL users will share the exact same friends list!

9. PRACTICE

Task: Add a `logout` method to the `User.prototype` from the example above.

Prototype chain

1. CONCEPT

> **The Prototype Chain is the series of links JavaScript traverses to find a requested property or method, continuing upward until it hits 'null'.**

2. SIMPLE ANALOGY

If a child doesn't know a recipe, they ask their parent. If the parent doesn't know, they ask the grandparent. If the grandparent doesn't know, the search fails. This upward search is the Prototype Chain.

3. SYNTAX

```
obj.__proto__ === Parent.prototype
```

4. EASY CODE

```
const grandparent = { surname: "Saha" };

const parent = { __proto__: grandparent, occupation: "Engineer" };

const child = { __proto__: parent, name: "Dithsa" };

// Searching up the chain!
console.log(child.name);       // Found on child
console.log(child.occupation); // Found on parent
console.log(child.surname);    // Found on grandparent
console.log(child.salary);     // Hits null -> undefined
```

5. OUTPUT

```
Output:
Dithsa
Engineer
Saha
undefined
```

6. EXPLANATION

The `__proto__` property is the secret link. When we ask for `child.surname`, JS checks `child`. Not there. It follows the link to `parent`. Not there. It follows the link to `grandparent`. Found it!

7. REAL-WORLD USE

> Understanding the prototype chain is the absolute most critical concept for senior-level JavaScript interviews.

8. COMMON MISTAKES

> **Caution:** Modifying `__proto__` directly in production code. It is terrible for performance. Always use `Object.create()` or modern `class` inheritance instead.

9. PRACTICE

> **Task: What does the top of every prototype chain eventually point to? (Hint: It signifies 'nothing').**

Classes

1. CONCEPT

> **The ES6 'class' keyword provides a clean, modern syntax for creating objects and dealing with inheritance, acting as 'syntactic sugar' over prototypes.**

2. SIMPLE ANALOGY

A 'class' is just a shiny new paint job over the old factory machine (constructor functions). Underneath the hood, the engine still uses prototypes, but the code looks much more professional and familiar to Java/C++ developers.

3. SYNTAX

```
class ClassName {
// Class body
}
```

4. EASY CODE

```
class Product {
    // A clean wrapper for object creation
}

const item = new Product();
console.log(typeof Product); // Surprise!
```

5. OUTPUT

```
Output:
function
```

6. EXPLANATION

Notice that `typeof Product` prints `function`. The `class` keyword did not add a new data type to JavaScript. It is literally just a beautiful wrapper around the old Constructor Function system we learned earlier.

7. REAL-WORLD USE

> Modern React and backend Node.js applications use classes heavily to structure application services and database models.

8. COMMON MISTAKES

> **Caution:** Forgetting that Classes are NOT hoisted. You cannot instantiate `new Product()` on line 1 if the `class Product` is defined on line 5.

9. PRACTICE

> **Task: Define a class called `Laptop`.**

Constructors

1. CONCEPT

> **The constructor is a special method inside a class that runs automatically the exact millisecond a new object is created.**

2. SIMPLE ANALOGY

The constructor is the initialization sequence of a robot. The moment you power it on for the first time, it runs the setup program to set its name, color, and starting battery level.

3. SYNTAX

```
class Name {
constructor(params) {
// Setup data
}
}
```

4. EASY CODE

```
class GameCharacter {
    // This runs the moment 'new' is used
    constructor(name, powerLevel) {
        this.name = name;
        this.powerLevel = powerLevel;
        this.isAlive = true; // Default value
        console.log(`Character ${this.name} spawned!`);
    }
}

const player1 = new GameCharacter("Ninja", 9000);
console.log(player1.powerLevel);
```

5. OUTPUT

```
Output:
Character Ninja spawned!
9000
```

6. EXPLANATION

We pass 'Ninja' and '9000' directly into the `new GameCharacter()` parentheses. These fall directly into the `constructor` parameters. We map them using `this`, establishing the object's properties.

7. REAL-WORLD USE

> Initializing a user session object by passing their auth token into the constructor the moment they log in.

8. COMMON MISTAKES

> **Caution:** Writing `Constructor()` instead of `constructor()`. It is completely lowercase. Also, you can only have exactly ONE constructor per class.

9. PRACTICE

> **Task: Write a class `Dog` with a constructor that takes `breed` and sets `this.breed`.**

Instance methods

1. CONCEPT

> **Instance methods are functions defined inside a class that belong to the prototype, making them accessible to all objects (instances) created from that class.**

2. SIMPLE ANALOGY

If the constructor defines the car's paint color (data), instance methods define the car's ability to drive, brake, and honk (behaviors).

3. SYNTAX

```
class Name {
methodName() { /* logic */ }
}
```

4. EASY CODE

```
class BankAccount {
    constructor(owner, balance) {
        this.owner = owner;
        this.balance = balance;
    }

    // Instance Method (Automatically added to the Prototype)
    deposit(amount) {
        this.balance += amount;
        console.log(`Deposited $${amount}. New balance: $${this.balance}`);
    }
}

const account1 = new BankAccount("Dithsa", 100);
account1.deposit(50);
```

5. OUTPUT

```
Output:
Deposited $50. New balance: $150
```

6. EXPLANATION

Notice we didn't write `function deposit()`. Inside a class, we just write the name of the method directly. The engine magically places this method onto the Prototype, saving memory.

7. REAL-WORLD USE

> A `VideoPlayer` class having instance methods like `play()`, `pause()`, and `seek()`.

8. COMMON MISTAKES

> **Caution:** Putting commas between methods. In a standard Object `{}`, methods are separated by commas. In a `class {}`, there are NO commas between methods.

9. PRACTICE

> **Task: Add a `withdraw(amount)` method to the `BankAccount` class.**

Static methods

1. CONCEPT

> **Static methods belong to the Class blueprint itself, NOT to the objects generated from it. They cannot be called on individual instances.**

2. SIMPLE ANALOGY

If a `Car` class builds cars, `drive()` is an instance method because individual cars drive. However, `Car.compareSpeed(car1, car2)` is a Static method. It belongs to the factory, not to the car itself.

3. SYNTAX

```
class Name {
static methodName() { }
}
```

4. EASY CODE

```
class MathHelper {
    // Static method belongs to the class blueprint
    static addMultiple(a, b) {
        return a + b;
    }
}

// ❌ ERROR: Instances cannot use static methods
// const math = new MathHelper();
// math.addMultiple(5, 5);

// ✅ CORRECT: Call it directly on the Class name
const result = MathHelper.addMultiple(10, 20);
console.log("Static Result:", result);
```

5. OUTPUT

```
Output:
Static Result: 30
```

6. EXPLANATION

The `static` keyword locks the method to the blueprint. We never have to use `new` to access it. We just call it directly on the capitalized class name `MathHelper`.

7. REAL-WORLD USE

> The built-in `Math.random()` or `Date.now()` are classic examples of static methods. You don't have to write `new Math()`, you just call the class.

8. COMMON MISTAKES

> **Caution:** Trying to use `this` inside a static method to access instance properties. Static methods have no idea what `this` is, because they aren't attached to an object.

9. PRACTICE

> **Task: Create a class `Calculator` with a static method `multiply(a,b)`.**

Getters/setters

1. CONCEPT

> **Getters and Setters are special methods that look like normal properties when accessed. They allow you to intercept data being read or written to add security or formatting.**

2. SIMPLE ANALOGY

A Getter is a receptionist. You don't grab the files from the back room yourself; you ask the receptionist, and they fetch it. A Setter is a bouncer. Before you are allowed to change the VIP list, the bouncer checks if your data is valid.

3. SYNTAX

```
get propName() { return data; }
set propName(value) { /* check */ }
```

IT STUDY HUB

4. EASY CODE

```
class Temperature {
    constructor(celsius) {
        this.celsius = celsius;
    }

    // GETTER: Format data on the fly
    get fahrenheit() {
        return this.celsius * 1.8 + 32;
    }

    // SETTER: Validate data before saving
    set setCelsius(value) {
        if (value < -273) {
            console.log("Error: Below absolute zero!");
            return;
        }
        this.celsius = value;
    }
}

const temp = new Temperature(25);
// Notice we access getters like properties, NO parentheses ()!
console.log(temp.fahrenheit);

temp.setCelsius = -300; // Triggers the setter validation
```

5. OUTPUT

```
Output:
77
Error: Below absolute zero!
```

6. EXPLANATION

Getters and setters disguise functions as properties. `temp.fahrenheit` calculates the math instantly without needing to call `temp.fahrenheit()`. `temp.setCelsius = -300` intercepts the assignment and blocks it.

7. REAL-WORLD USE

> Intercepting a password update to ensure it meets complexity requirements before actually saving it to the object.

8. COMMON MISTAKES

> **Caution:** Naming a setter the exact same thing as the property it modifies (e.g., `set celsius(value) { this.celsius = value; }`). This creates an infinite loop that crashes the browser instantly.

9. PRACTICE

> **Task: Create a getter `fullName` that combines `this.firstName` and `this.lastName`.**

Inheritance

1. CONCEPT

> **Inheritance is the ability of one class (Child) to automatically copy all the properties and methods of another class (Parent), reducing duplicate code.**

2. SIMPLE ANALOGY

If you design a 'Vehicle' blueprint that has wheels and an engine, you don't need to rebuild those things when designing a 'Motorcycle' blueprint. The Motorcycle inherits the Vehicle blueprint and simply adds handlebars.

3. SYNTAX

```
class Child extends Parent { }
```

4. EASY CODE

```
// 1. The Parent Class
class Animal {
    breathe() {
        console.log("Inhaling oxygen...");
    }
}

// 2. The Child Class (Inherits from Animal)
class Bird extends Animal {
    fly() {
        console.log("Soaring through the sky!");
    }
}

const eagle = new Bird();
eagle.breathe(); // Inherited from Animal!
eagle.fly();     // Native to Bird!
```

5. OUTPUT

```
Output:
Inhaling oxygen...
Soaring through the sky!
```

6. EXPLANATION

The `extends` keyword links the prototypes. The `eagle` object checks itself for `breathe()`. It doesn't have it, so it looks up the prototype chain to `Animal` and executes it perfectly.

7. REAL-WORLD USE

A Base `User` class (login/logout) inherited by `AdminUser`, `ModeratorUser`, and `GuestUser`, each adding their own specific permissions.

8. COMMON MISTAKES

Caution: Creating excessively deep inheritance trees (Grandparent -> Parent -> Child -> Grandchild). It becomes a nightmare to debug. Prefer Composition over deep Inheritance.

9. PRACTICE

Task: Create a class `Vehicle` with `drive()`. Create a `Car` that `extends Vehicle`.

extends

1. CONCEPT

The 'extends' keyword is specifically used in class declarations to create a class that is a child of another class.

2. SIMPLE ANALOGY

The `extends` keyword is the physical pipeline connecting two factories. It ensures that whatever the parent factory produces, the child factory automatically has access to.

3. SYNTAX

```
class Admin extends User { }
```

4. EASY CODE

```
class Shape {
    constructor(color) {
        this.color = color;
    }
    describe() { return `A ${this.color} shape.`; }
}

// Circle 'extends' Shape
class Circle extends Shape {
    // Circle gets the constructor and describe() automatically!
}

const myCircle = new Circle("Red");
console.log(myCircle.describe());
```

5. OUTPUT

```
Output:
A Red shape.
```

6. EXPLANATION

Because we didn't write a constructor in `Circle`, it automatically stole the constructor from `Shape`. The 'Red' argument bypassed `Circle` and went straight to the `Shape` constructor.

7. REAL-WORLD USE

> React heavily uses this: `class App extends React.Component`. You instantly inherit thousands of lines of UI rendering logic from the React library.

8. COMMON MISTAKES

> **Caution:** Trying to `extend` multiple classes at once (e.g., `class Child extends Father, Mother`). JavaScript explicitly forbids multiple inheritance. You can only extend ONE class.

9. PRACTICE

> **Task: Write `class Teacher extends Employee {}`.**

super

1. CONCEPT

> **The 'super' keyword is used to access and call functions on an object's parent class. It is mandatory in a child class's constructor.**

2. SIMPLE ANALOGY

If a child inherits a house from their parents but wants to add a swimming pool, they must call the contractor (`super`) to build the base house FIRST before they can add the pool.

3. SYNTAX

```
super(parentArgs);
super.parentMethod();
```

4. EASY CODE

```
class Employee {
    constructor(name) {
        this.name = name;
    }
}

class Manager extends Employee {
    constructor(name, department) {
        // MUST call super() first to build the 'Employee' foundation
        super(name);

        // Only AFTER super() can we use 'this'
        this.department = department;
    }
}

const boss = new Manager("Mithila", "IT");
console.log(`${boss.name} runs ${boss.department}`);
```

5. OUTPUT

```
Output:
Mithila runs IT
```

6. EXPLANATION

If a child class has its own `constructor`, you are physically forbidden from using `this` until you call `super()`. `super(name)` teleports the name up to the `Employee` constructor to be mapped.

7. REAL-WORLD USE

Passing shared configuration data up to a base database connection class before initializing specific table logic.

8. COMMON MISTAKES

Caution: Trying to use `this.department = department` before calling `super()`. The engine will crash with a 'Must call super constructor' error.

9. PRACTICE

Task: If `Parent` takes `age`, write the first line of a `Child` constructor taking `(name, age)`.

Encapsulation

1. CONCEPT

Encapsulation is the bundling of data and the methods that act on that data into a single unit, while restricting direct outside access to some of the components.

2. SIMPLE ANALOGY

An ATM is encapsulated. You cannot directly reach into the vault and grab money. You must use the keypad (the interface). The keypad checks your pin, validates the amount, and then dispenses the cash safely.

3. SYNTAX

```
// Achieved using closures or ES2022 private fields (#)
```

4. EASY CODE

```
class BankSafe {
    constructor() {
        // Old convention: '_' means "Please don't touch this directly"
        this._balance = 1000;
    }

    // The public interface
    withdraw(amount) {
        if (amount > this._balance) {
            console.log("Denied: Insufficient funds.");
        } else {
            this._balance -= amount;
            console.log(`Dispensing $${amount}`);
        }
    }
}

const mySafe = new BankSafe();
mySafe.withdraw(5000); // Prevented safely by the method!
```

5. OUTPUT

```
Output:
Denied: Insufficient funds.
```

6. EXPLANATION

By forcing users to go through the `withdraw()` method, we encapsulate the logic. However, because `_balance` is just a naming convention, a bad developer could still manually type `mySafe._balance = 0` and bypass the security.

7. REAL-WORLD USE

Hiding API keys or internal tracking IDs inside a class so that other parts of the application cannot accidentally overwrite them.

8. COMMON MISTAKES

Caution: Assuming the underscore `_` actually protects the variable. It does not. It is merely a 'gentleman's agreement' among developers to leave it alone.

9. PRACTICE

Task: Explain why accessing variables through methods is safer than accessing them directly.

Polymorphism

1. CONCEPT

Polymorphism (meaning 'many forms') allows different classes to have methods with the exact same name, but execute completely different logic.

2. SIMPLE ANALOGY

If you tell a dog to 'Speak', it barks. If you tell a cat to 'Speak', it meows. If you tell a human to 'Speak', they say hello. The command is the exact same ('Speak'), but the execution takes many forms.

3. SYNTAX

```
// Method Overriding
```

4. EASY CODE

```
class Notification {
    send() { console.log("Sending a generic alert."); }
}

class EmailNotification extends Notification {
    // OVERRIDING the parent method
    send() { console.log("Sending an Email..."); }
}

class SMSNotification extends Notification {
    // OVERRIDING the parent method
    send() { console.log("Sending a Text Message..."); }
}

const alerts = [new EmailNotification(), new SMSNotification()];

// We can loop them and call the same method seamlessly!
alerts.forEach(a => a.send());
```

5. OUTPUT

```
Output:
Sending an Email...
Sending a Text Message...
```

6. EXPLANATION

Because both children inherited `send()`, they replaced the generic parent logic with their own specific logic. We can now loop through a massive array of different objects and blindly call `.send()` on all of them, knowing they will handle their own logic perfectly.

7. REAL-WORLD USE

A payment processing system where `.process()` acts differently if the object is `CreditCard`, `PayPal`, or `Crypto`.

8. COMMON MISTAKES

Caution: Forgetting that polymorphism relies on identical method names. If the SMS class used `sendSMS()` instead of `send()`, the loop would crash.

9. PRACTICE

Task: Create a `Shape` class with `draw()`. Create `Square` overriding `draw()`.

Composition vs inheritance

1. CONCEPT

Inheritance designs objects based on 'what they ARE' (A Dog is an Animal). Composition designs objects based on 'what they DO' by mixing and matching independent behaviors.

2. SIMPLE ANALOGY

Inheritance is a rigid family tree. Composition is building with LEGOs. Instead of saying 'A FlyingCar is a Car that inherits from a Bird', you just stick the 'Drive' LEGO piece and the 'Fly' LEGO piece onto a single object.

3. SYNTAX

```
const obj = Object.assign({}, canDrive, canFly);
```

4. EASY CODE

```
// Behaviors (LEGO pieces)
const canWalk = {
    walk() { console.log(`${this.name} is walking.`); }
};

const canFly = {
    fly() { console.log(`${this.name} is flying.`); }
};

// Creating an object using Composition
const createDuck = (name) => {
    // Combine the behaviors into a new object
    return Object.assign({ name: name }, canWalk, canFly);
};

const myDuck = createDuck("Donald");
myDuck.walk();
myDuck.fly();
```

5. OUTPUT

```
Output:
Donald is walking.
Donald is flying.
```

6. EXPLANATION

Instead of using `class Duck extends Bird`, we just merged raw objects together using `Object.assign`. This completely bypasses the strict prototype chain and prevents the 'Gorilla Banana' problem (requesting a banana, but inheriting a gorilla holding it).

7. REAL-WORLD USE

> React explicitly advises using Composition over Inheritance to build UI components, passing behaviors dynamically as 'props'.

8. COMMON MISTAKES

> **Caution:** Using deep 7-level inheritance trees. When requirements change, changing the top parent breaks all 7 children. Composition is flat and unbreakable.

9. PRACTICE

> **Task: Explain the difference between 'IS A' (Inheritance) and 'HAS A' (Composition).**

Private fields

1. CONCEPT

> **ES2022 introduced true Private Fields using the hashtag (#) symbol, allowing developers to create data inside a class that is completely impenetrable from the outside.**

2. SIMPLE ANALOGY

Unlike the gentleman's agreement of the underscore `_`, Private Fields are a titanium bank vault. If anything outside the class attempts to read or write the data, the JavaScript engine violently blocks the action.

3. SYNTAX

```
class Name { #privateVar = 10; }
```

4. EASY CODE

```
class SecureVault {
    // Declare the private field with a '#'
    #password;

    constructor(pass) {
        this.#password = pass; // Save to the private field
    }

    checkPassword(attempt) {
        return attempt === this.#password;
    }
}

const vault = new SecureVault("TopSecret123");

console.log(vault.checkPassword("TopSecret123")); // true
// console.log(vault.#password); // ❌ SYNTAX ERROR: Private field!
```

5. OUTPUT

```
Output:
true
```

6. EXPLANATION

The `#password` variable is physically inaccessible from the global scope. The only way to interact with it is through the public `checkPassword` method which lives inside the same class boundaries.

7. REAL-WORLD USE

Securing sensitive user data (social security numbers, session tokens) inside client-side applications.

8. COMMON MISTAKES

Caution: Trying to create a private field dynamically in the constructor without declaring it at the top of the class first. It MUST be declared at the class level.

9. PRACTICE

Task: Write a class `Person` with a private field `#age`.

Advanced this

1. CONCEPT

The value of 'this' is lost when methods are separated from their objects (like when passed as callbacks). We must secure it using arrow functions or binding.

2. SIMPLE ANALOGY

If an employee (method) leaves the office (object) to go do a job elsewhere (callback), they forget who they work for (`this` becomes undefined). We must pin a nametag on them before they leave.

3. SYNTAX

```
setTimeout(() => this.method(), 1000);
```

4. EASY CODE

```
class Timer {
    constructor() {
        this.message = "Time is up!";
    }

    startBroken() {
        // The standard function creates its own new 'this' (undefined)
        setTimeout(function() {
            console.log("Broken:", this.message);
        }, 500);
    }

    startFixed() {
        // Arrow functions inherit 'this' from their parent!
        setTimeout(() => {
            console.log("Fixed:", this.message);
        }, 500);
    }
}

const myTimer = new Timer();
// myTimer.startBroken(); // ❌ Logs undefined
myTimer.startFixed();     // ✅ Logs "Time is up!"
```

5. OUTPUT

```
Output:
Fixed: Time is up!
```

6. EXPLANATION

In `startBroken`, `setTimeout` executes the callback in the global window context, ripping it away from the `Timer` object. Because Arrow Functions lack their own `this`, they act transparently, allowing the code to see through them to the `Timer` class's `this`.

7. REAL-WORLD USE

> Fixing React component event handlers where passing a method to an `onClick` attribute causes it to lose its context.

8. COMMON MISTAKES

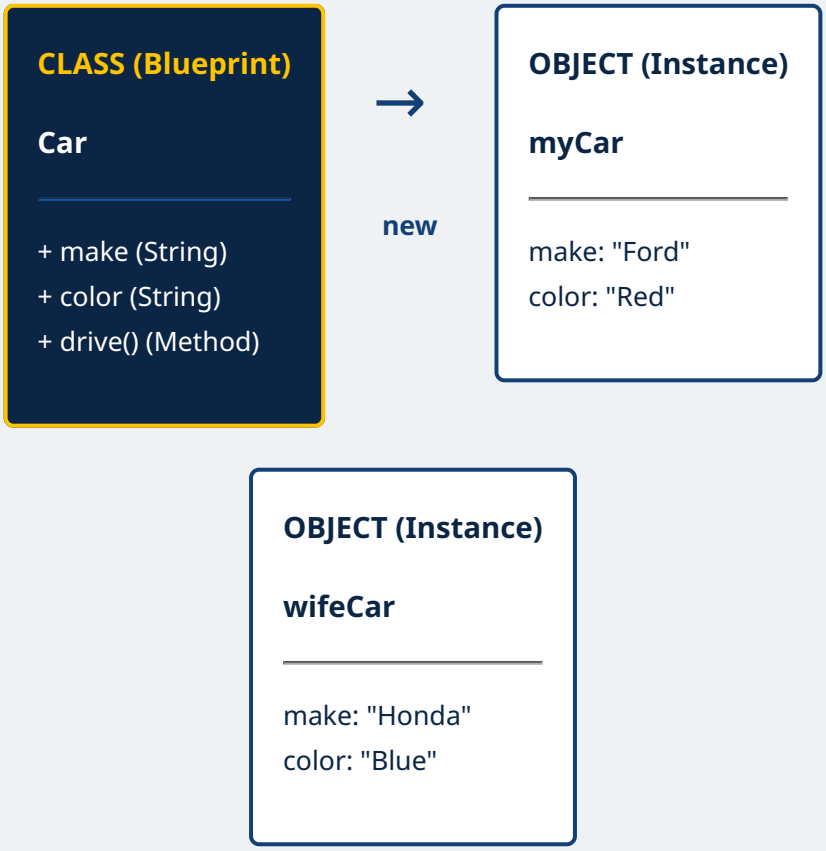
> **Caution:** Using arrow functions everywhere blindly. Arrow functions are great for securing `this` in callbacks, but terrible as top-level object methods.

9. PRACTICE

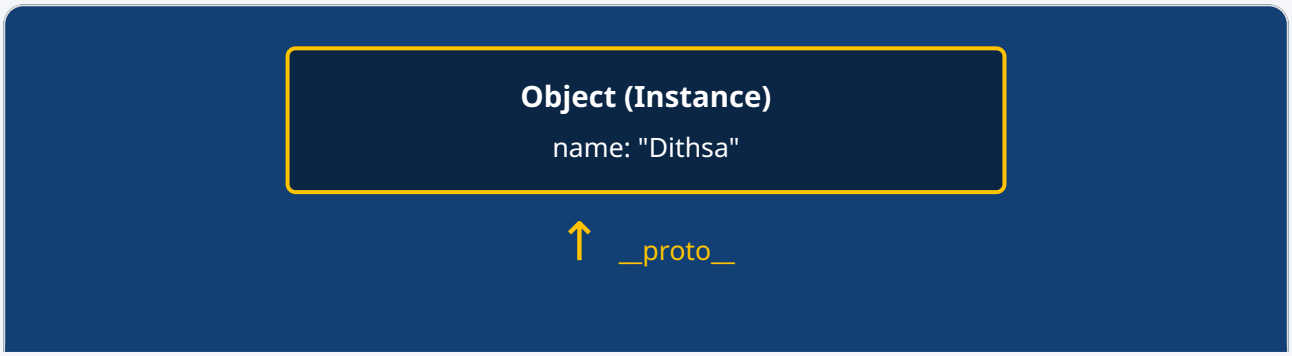
> **Task: Why does an arrow function solve the 'lost this' problem in callbacks?**

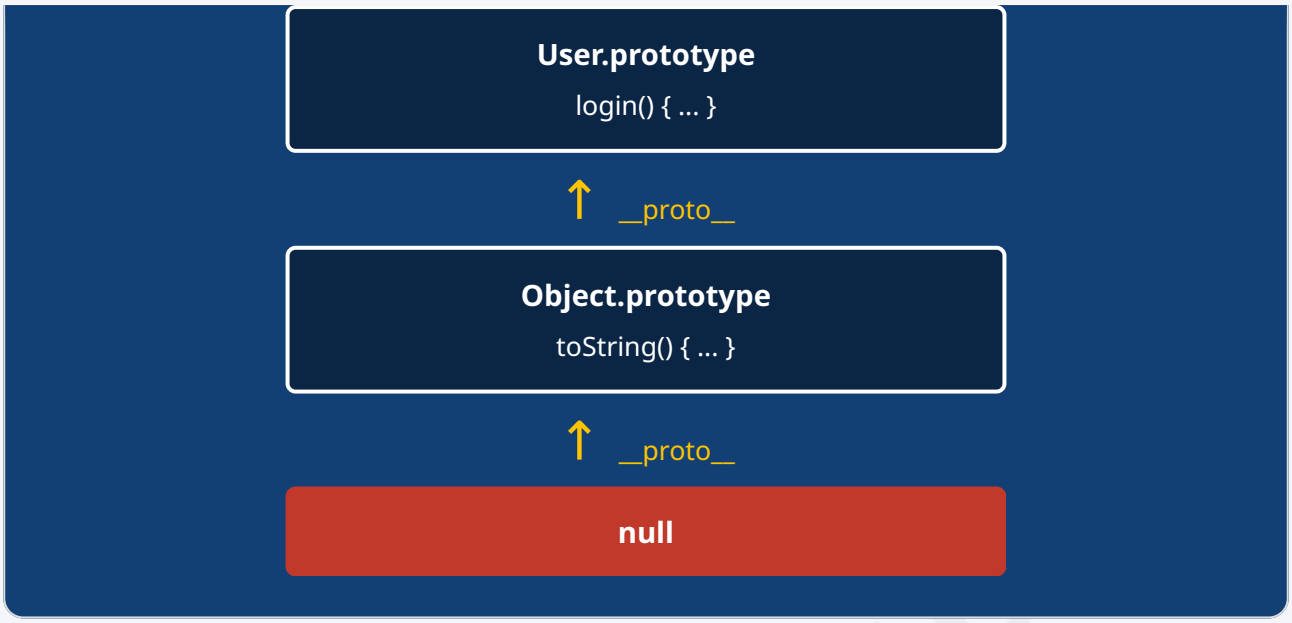
Visual Architecture & Workflows

1. CLASS VS OBJECT IMPLEMENTATION

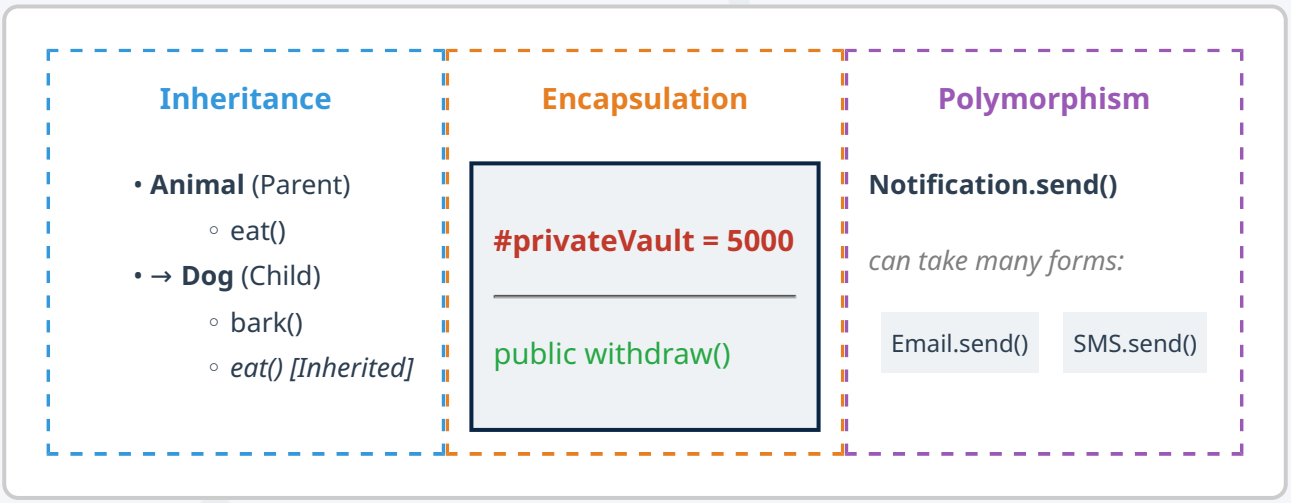


2. THE PROTOTYPE CHAIN





3. INHERITANCE, ENCAPSULATION & POLYMORPHISM



Hands-on Project: Library Management System

We will build a robust Object-Oriented system. We will create a base Book class utilizing encapsulation to protect data. We will create a `DigitalBook` child class utilizing inheritance. Finally, we will build a `Library` management class to orchestrate the objects.

IT STUDY HUB

THE ARCHITECTURE (APP.JS)

IT STUDY HUB

```
"use strict";

// 1. BASE CLASS with Encapsulation
class Book {
    #isAvailable; // Private Field

    constructor(title, author) {
        this.title = title;
        this.author = author;
        this.#isAvailable = true; // Default state
    }

    // Getter to safely read private data
    get status() {
        return this.#isAvailable ? "Available" : "Borrowed";
    }

    // Instance Method modifying private data securely
    borrow() {
        if (this.#isAvailable) {
            this.#isAvailable = false;
            console.log(`Success: You borrowed '${this.title}'.`);
        } else {
            console.log(`Denied: '${this.title}' is already checked out.`);
        }
    }

    returnBook() {
        this.#isAvailable = true;
        console.log(`Returned: '${this.title}'. Thank you!`);
    }
}

// 2. CHILD CLASS demonstrating Inheritance & Polymorphism
class DigitalBook extends Book {
    constructor(title, author, fileSizeMB) {
        super(title, author); // Build the base book first!
        this.fileSizeMB = fileSizeMB;
    }
```

```
    // OVERRIDING the borrow method (Polymorphism)
    borrow() {
        console.log(`Downloading '${this.title}' (${this.fileSizeMB}
MB)...`);
        // Digital books are infinitely available, no need to touch
#isAvailable
    }
}

// 3. SYSTEM MANAGER CLASS
class Library {
    constructor(name) {
        this.name = name;
        this.inventory = [];
    }

    addBook(bookObj) {
        this.inventory.push(bookObj);
        console.log(`[${this.name}] Added: ${bookObj.title}`);
    }

    showInventory() {
        console.log(`\n--- ${this.name} Inventory ---`);
        this.inventory.forEach(book => {
            console.log(`${book.title} by ${book.author} | Status: $
{book.status}`);
        });
        console.log("-------------------------\n");
    }
}

// --- SYSTEM EXECUTION ---

const cityLibrary = new Library("Central City Library");

// Instantiating Objects
const book1 = new Book("The Great Gatsby", "F. Scott Fitzgerald");
const book2 = new DigitalBook("JavaScript Mastery", "IT Study Hub", 15.5);

// Orchestrating through the Manager
```

```
cityLibrary.addBook(book1);
cityLibrary.addBook(book2);

cityLibrary.showInventory();

// Interacting securely with Encapsulated Objects
book1.borrow();
book1.borrow(); // Should fail
book2.borrow(); // Polymorphic behavior!

cityLibrary.showInventory();
```

Code Explanation:

- **Encapsulation:** The `#isAvailable` flag is completely invisible to the outside. Hackers cannot manually change a book to 'Available' without formally calling the `returnBook()` method.
- **Inheritance:** `DigitalBook` steals the title and author logic from `Book` using `super`, preventing duplicate code.
- **Polymorphism:** When we call `.borrow()` on a physical book, it triggers security logic. When we call `.borrow()` on a digital book, it triggers a download message instead.

IT STUDY HUB

Module 9 Assessment & Practice

Knowledge is solidified through repetition. Complete these sections to master the module.

15 BEGINNER EXERCISES

1. **Explain the difference between Procedural programming and OOP.**

2. **Write an empty class named `Library`.**

3. **Instantiate two separate objects from the `Library` class.**

4. **Explain the difference between a Class and an Object.**

5. **Create a constructor function for `Car(brand)`. Use `new` to build one.**

6. **Why is it more memory-efficient to put methods on the Prototype rather than the constructor?**

7. **Explain the Prototype Chain.**

8. **Write a `constructor` inside a class that takes `name` and sets `this.name`.**

9. **Add an instance method `speak()` to the class.**

IT STUDY HUB

10. Write a `static` method `info()` on the class. Call it without using `new`.

11. Use `get` to create a getter that returns a formatted string.

12. Create `class Dog extends Animal`. What does `extends` do?

13. Write a child constructor that calls `super(name)`.

14. What does the `#` symbol do in front of a class property?

15. Explain Polymorphism in your own words.

IT STUDY HUB

20 VIVA QUESTIONS & ANSWERS

Question	Answer
What is the primary benefit of Object-Oriented Programming?	It bundles data and behavior into single entities, making large codebases modular, reusable, and easier to maintain.
What does the `new` keyword actually do behind the scenes?	It creates a blank object, binds `this` to the new object, links the prototype, and automatically returns the object.
What is a Prototype?	An internal object that other objects delegate to, serving as a fallback for shared properties and methods.
What happens if a property is not found on an object or its prototype?	The engine traverses the prototype chain until it hits `null`, at which point it evaluates to `undefined`.
Are JavaScript classes true classes like in Java or C++?	No, they are 'syntactic sugar' acting as a cleaner wrapper over JavaScript's existing prototype-based inheritance system.
Can you call a `static` method on an instantiated object?	No, static methods belong exclusively to the class blueprint itself, not the instances.
What is the purpose of a Getter?	To intercept property access, allowing you to compute or format a value dynamically without the user needing to call a function.
What happens if a Child class has a constructor but forgets to call `super()`?	The JavaScript engine throws a ReferenceError because `this` cannot be initialized without the parent constructor.

What is Encapsulation?	The hiding of internal state and requiring all interaction to be performed through an object's public methods.
Why was the `_` prefix used before ES2022?	As a naming convention to warn other developers that a property was intended to be private, though it offered no actual security.
What does the `#` prefix do?	It enforces true privacy at the engine level, making the field physically inaccessible from outside the class block.
What is Polymorphism?	The ability of different objects to share the same method name, but execute their own overridden, specific logic.
What is the 'Gorilla Banana' problem in deep Inheritance?	When you want a single piece of functionality (a banana), but you are forced to inherit the entire complex parent class (the gorilla holding it and the entire jungle).
How does Composition differ from Inheritance?	Inheritance defines an object by what it IS. Composition builds an object by combining what it can DO (merging independent behaviors).
How does an Arrow function handle `this`?	It does not create its own `this` context; it transparently inherits `this` from the surrounding lexical scope.
Why do standard functions lose `this` when passed as callbacks (like in setTimeout)?	Because they are executed by the global window or the callback queue, stripping them from their original object context.
	No, JavaScript strictly allows only one constructor per class.

Can you have multiple `constructor()` methods in a single JavaScript class?	
What does `Object.assign()` do?	It copies the values of all enumerable own properties from one or more source objects to a target object (used heavily in Composition).
Is OOP the only way to write modern JavaScript?	No, many developers prefer Functional Programming (FP), and React heavily shifted from OOP (Class components) to FP (Hooks).
If `Array.prototype.map` is modified, what happens?	Every single array in the entire application instantly inherits the modification, which is why altering built-in prototypes is heavily discouraged.

IT STUDY

10 INTERVIEW QUESTIONS

These questions test your deeper conceptual understanding. Practice answering them verbally.

- **Explain the architectural difference between Classical Inheritance (Java/C++) and Prototypal Inheritance (JavaScript).**
- **Walk me through the exact execution steps of the Prototype Chain when `obj.toString()` is called on a standard object.**
- **Why were Classes introduced in ES6 if they are just syntactic sugar over Constructor Functions? What architectural problems did they solve?**
- **Defend Composition over Inheritance. Provide a specific data modeling scenario where Inheritance breaks down but Composition succeeds.**
- **Explain how you would implement true Encapsulation in pre-ES2022 JavaScript (before Private `#` fields existed) using Closures.**
- **How does the `super` keyword route arguments in a multi-level inheritance tree (e.g., Grandchild -> Child -> Parent)?**
- **Describe the memory implications of defining a method inside a `constructor` block versus defining it as a Class Instance Method.**
- **How does Polymorphism reduce the need for complex `switch` or `if/else` control flows when iterating over arrays of mixed object types?**
- **Explain why passing an object method as an event listener callback (`btn.addEventListener('click', obj.method)`) inherently strips its `this` context, and how to fix it.**

IT STUDY HUB

- **If everything in JavaScript is technically an object (including functions and arrays), explain the conceptual difference between a primitive type and a reference type.**

IT STUDY HUB

Module Summary

In this module, you transcended basic scripting and entered the realm of Enterprise Software Architecture. You abandoned procedural code to embrace **Object-Oriented Programming (OOP)**. You learned how to model the physical world by designing **Classes** that fuse isolated data with actionable methods.

You dissected JavaScript's underlying memory structure, revealing how the **Prototype Chain** shares resources to optimize RAM. You utilized the `extends` and `super` keywords to build deep **Inheritance** trees, passing traits down to child classes. Furthermore, you applied **Polymorphism** to override behaviors dynamically, weighed the modular benefits of **Composition**, and constructed impenetrable data vaults using ES2022 Private Fields to guarantee perfect **Encapsulation**.

WHAT COMES NEXT?

MODULE 10 — ERROR HANDLING & DEBUGGING

You now possess the capability to build massive, highly complex, asynchronous, object-oriented applications. But with immense complexity comes inevitable failure. In Module 10, we will learn how to handle destruction. You will master advanced Error Handling, utilizing custom error classes, global stack trace reading, sophisticated `try/catch/finally` topologies, and deep browser DevTools debugging to ensure your applications survive catastrophic runtime failures seamlessly.